Автономная некоммерческая образовательная организация высшего образования

«Научно-технологический университет «Сириус»»

Лабораторная работа № 1

«Ввод-вывод дискретных сигналов»

Вариант 2

По дисциплине: Основы программирования для микроэлектроники

Выполнил:

Студент

Мегерян Сергей Сергеевич

Проверил:

_____

ФТ Сириус, 2026 г.

Тема

Формирование динамической светодиодной индикации на основе дискретных сигналов.

Цель работы

Изучить методы управления светодиодной линейкой и реализовать эффект бегущего огня с изменяемой скоростью по нажатию кнопок.

Оборудование

- Arduino Uno
- 6 светодиодов
- 6 резисторов 220 Ом
- 2 кнопки
- макетная плата
- соединительные провода

Схема подключения

- D8-D13 -> 220 Ом -> аноды светодиодов
- катоды светодиодов -> GND

- FAST: D2 -> кнопка -> GND
- SLOW: D3 -> кнопка -> GND

Описание алгоритма

Светодиоды подключены к младшим линиям Port B. В каждый момент времени включен только один светодиод. Через заданный интервал времени активный светодиод последовательно перемещается по линейке, после чего цикл повторяется. Кнопки изменяют длительность паузы между шагами, тем самым регулируя скорость эффекта. Для исключения ложных срабатываний применяется программный антидребезг.

Результат работы программы

После запуска программы на светодиодной линейке формируется бегущий огонь. Пользователь может изменять скорость движения в широких пределах с помощью двух кнопок.

Вывод

В работе была реализована динамическая индикация на дискретных выходах микроконтроллера и освоены приемы программного управления скоростью световых эффектов.

Таблица соединений

Элемент	Подключение

LED0-LED5	D8-D13 через резисторы 220 Ом
FAST	D2 -> кнопка -> GND
SLOW	D3 -> кнопка -> GND

Иллюстрации и скриншоты

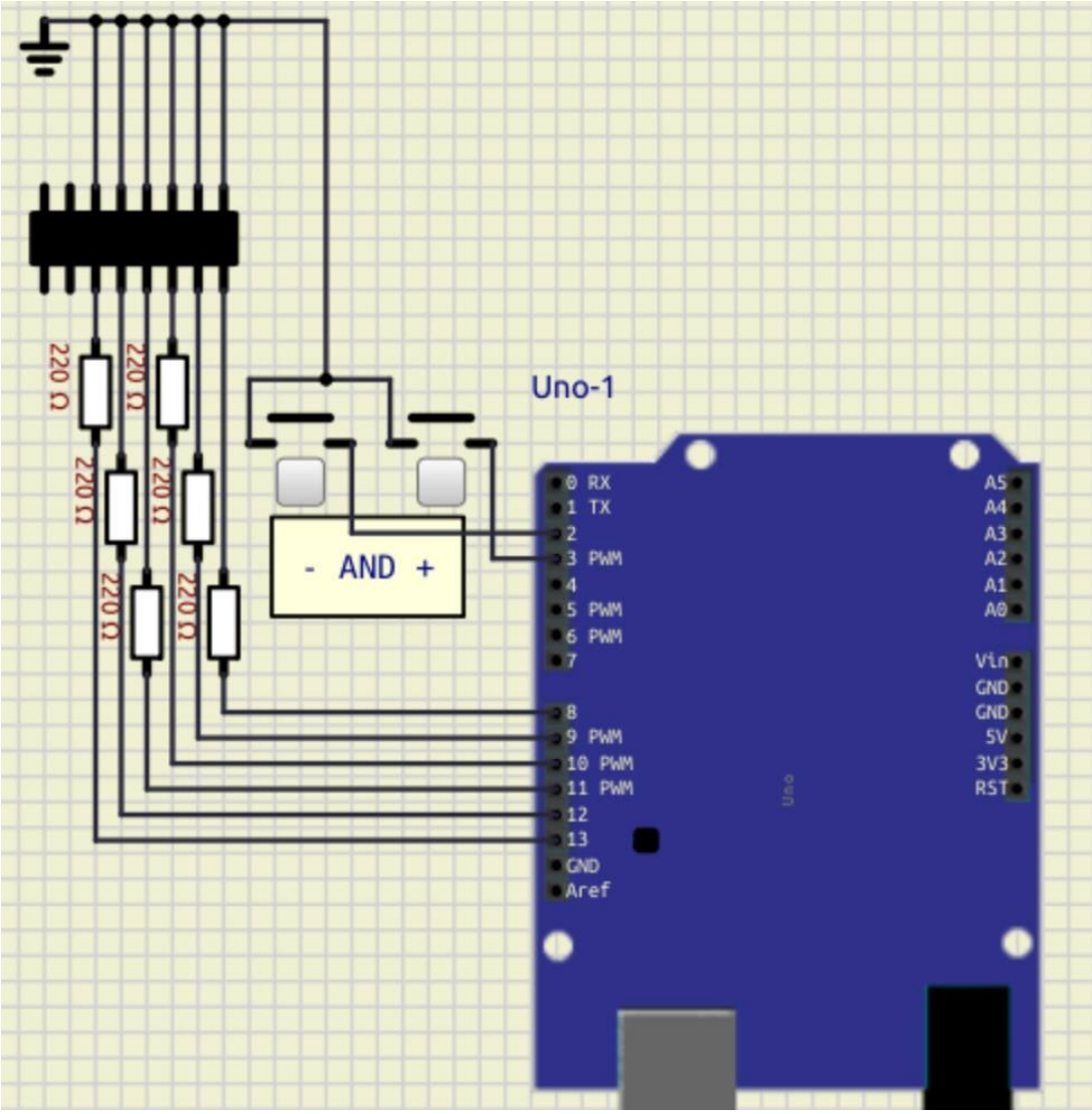

Рисунок 1 - Пример схемы подключения

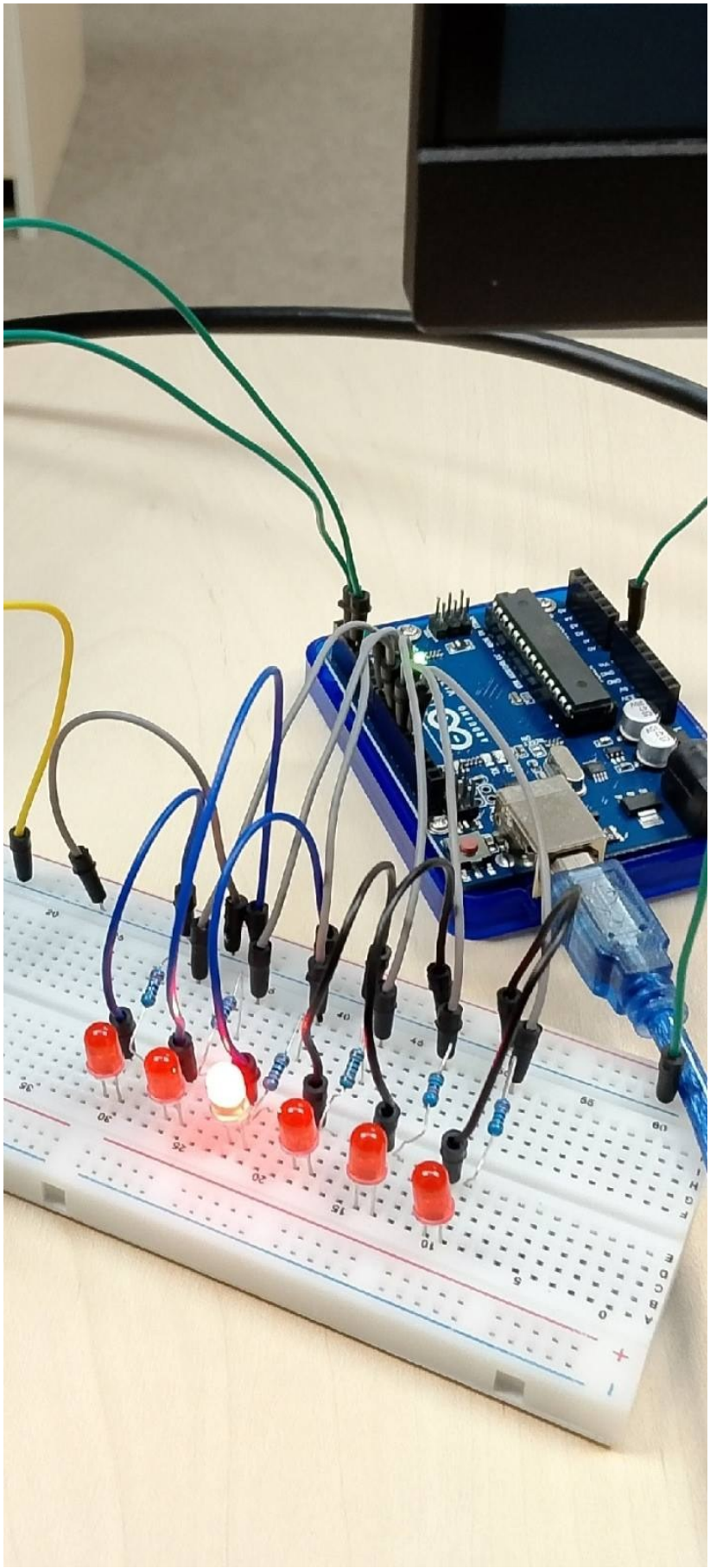

Рисунок 2 - Макет со светодиодной линейкой

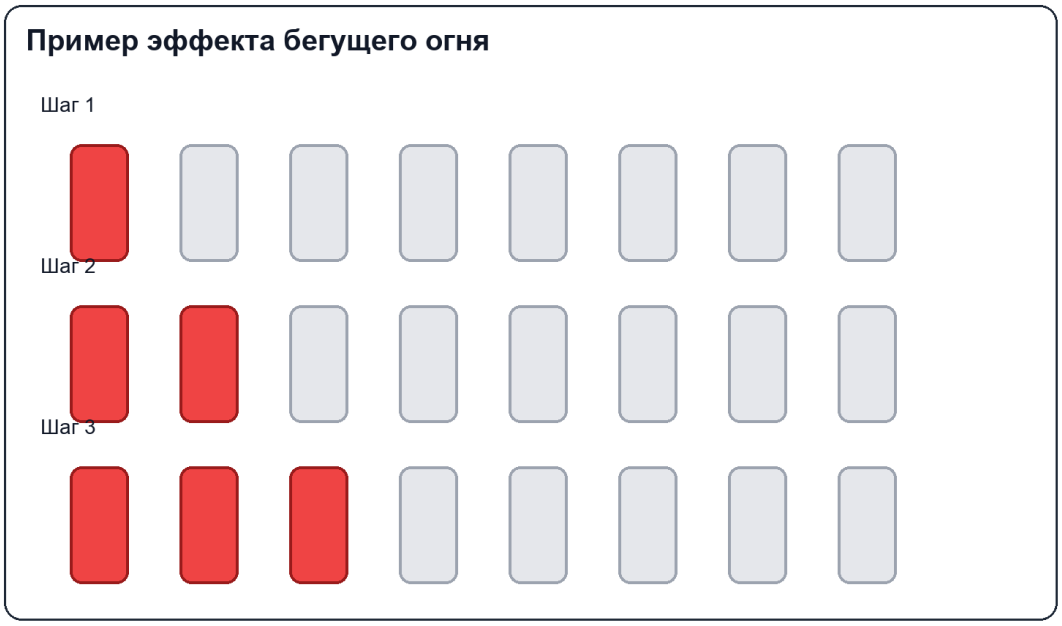

Рисунок 3 - Иллюстрация эффекта бегущего огня

Приложение. Листинг программы

lab1_variant2.ino

```
#include <Arduino.h>


const uint8_t FASTER_BUTTON_PIN = 2;
const uint8_t SLOWER_BUTTON_PIN = 3;
const uint8_t PORTB_LED_MASK = 0b00111111;
const unsigned long DEBOUNCE_DELAY_MS = 50;
const unsigned long MIN_STEP_DELAY_MS = 50;
const unsigned long MAX_STEP_DELAY_MS = 1000;
const unsigned long STEP_DELAY_DELTA_MS = 50;


bool lastFasterReading = HIGH;
bool fasterStableState = HIGH;
unsigned long fasterDebounceTime = 0;


bool lastSlowerReading = HIGH;
bool slowerStableState = HIGH;
unsigned long slowerDebounceTime = 0;


uint8_t currentLedIndex = 0;
unsigned long stepDelayMs = 250;
unsigned long lastStepTime = 0;


void outputRunningLight(uint8_t ledIndex) {
```

```
  PORTB = (PORTB & 0b11000000) | (1 << ledIndex);
}

void handleFasterButton() {
  bool reading = digitalRead(FASTER_BUTTON_PIN);

  if (reading != lastFasterReading) {
    fasterDebounceTime = millis();
    lastFasterReading = reading;
  }

  if ((millis() - fasterDebounceTime) > DEBOUNCE_DELAY_MS && reading !=
fasterStableState) {
    fasterStableState = reading;

    if (fasterStableState == LOW && stepDelayMs > MIN_STEP_DELAY_MS) {
      stepDelayMs -= STEP_DELAY_DELTA_MS;
      if (stepDelayMs < MIN_STEP_DELAY_MS) {
        stepDelayMs = MIN_STEP_DELAY_MS;
      }
    }
  }
}

void handleSlowerButton() {
  bool reading = digitalRead(SLOWER_BUTTON_PIN);

  if (reading != lastSlowerReading) {
    slowerDebounceTime = millis();
    lastSlowerReading = reading;
  }

  if ((millis() - slowerDebounceTime) > DEBOUNCE_DELAY_MS && reading !=
slowerStableState) {
    slowerStableState = reading;

    if (slowerStableState == LOW && stepDelayMs < MAX_STEP_DELAY_MS) {
      stepDelayMs += STEP_DELAY_DELTA_MS;
      if (stepDelayMs > MAX_STEP_DELAY_MS) {
        stepDelayMs = MAX_STEP_DELAY_MS;
      }
    }
  }
}

void setup() {
  DDRB |= PORTB_LED_MASK;
  outputRunningLight(currentLedIndex);

  pinMode(FASTER_BUTTON_PIN, INPUT_PULLUP);
  pinMode(SLOWER_BUTTON_PIN, INPUT_PULLUP);
}
```

```
void loop() {
  handleFasterButton();
  handleSlowerButton();

  unsigned long now = millis();
  if (now - lastStepTime >= stepDelayMs) {
    lastStepTime = now;
    currentLedIndex = (currentLedIndex + 1) % 6;
    outputRunningLight(currentLedIndex);
  }
}
```